

Tool-Use Summary

Santheesh S · Mystri Track B · Product 1 · 3 h 50 min · sivasandy509@gmail.com

I used AI and local tools under Mystri's transparency requirement. My submission stands on what a reviewer can replay: **python product1verify_product1.py** returning **PRODUCT 1 PASS**, 39 unit tests on unchanged pack **data/**, and the narrative in **DECISION.md** and **SOURCES.md**. I treat Composer as a fast draft layer; I own the decision, the rules, and the pass/fail gate.

I work with five habits on a timed build: keep the **human decision** and let AI handle drafts; **verify before trust** on every substantive code change; separate **file collection from follow-up policy**; **fail closed** on ambiguous rows such as R018; and state **honest limits** rather than overstating ROI or the owner's eight-hour claim.

Tools and how I governed their output

Tool	What I used it for	How I checked / corrected / rejected
Cursor — Composer	Scaffolded queue_engine.py , parts of integrated_model.py , test stubs, and doc structure. No separate paid LLM API for graded runs.	After edits: verify_product1.py and full unittest ; aligned with DATA_DICTIONARY.md ; conflicting fields routed to uncertain .
Python 3.10+	Loader, experiment.py , verify gate, 39 tests, dry-run JSON and trace.	PRODUCT 1 PASS ; baseline 13 vs rules 5 in test_pack_snapshot_baseline_thirteen_rules_five .
Manual analysis	Deduped event_id ; naive vs rules counts for DECISION.md .	Cross-checked CSVs and tests; did not treat owner 8 h/week as measured.
Vendor documentation	OneDrive/Dropbox file-request; Microsoft Learn share-files; Jotform for context.	Logged in SOURCES.md with dates, claims, and limits.

Division of responsibility

Role	Responsibility
Composer	Boilerplate, test skeletons, first-pass documentation.
Me	Rule semantics, evidence, Product 1 scope, SOURCES claims, pass/fail criteria.
Reject rule	If verify or tests fail, I fix or revert — I do not ship failing AI output.

Excluded from this submission

Category	Reason
Paid LLM APIs	Graded proof uses stdlib and local verify only.
Deployment / hosting	Dry-run assessment scope.
Live messaging	Simulated in trace and JSON only.
Editing pack data/	Verify assumes original Mystri CSVs.

Example I caught during implementation

Composer once emitted an invalid dataclass update in queue code. Unit tests failed immediately. I rejected that version, rewrote with **dataclasses.replace**, and re-ran the full suite and verify until **PRODUCT 1 PASS**. That is the standard I apply to any AI-generated change.

Step	Outcome
Problem	Invalid dataclass update in generated code.
Signal	Unit tests failed.
Action	Rejected assistant output; fixed with dataclasses.replace.
Proof	Full unittest + verify → PRODUCT 1 PASS.

Where this lives in the repo

Artifact	Role
DECISION.md	Decision note with claim, alternatives, counterargument.
HANDOVER.md	Run steps, evidence, tool judgment.
SOURCES.md	External references and limitations.
product1/verify_product1.py	Single reviewer gate.

github.com/santheesh15/-.mystri-assessment · branch [cursor/track-b-submission-edb7](#) · [track-b/docs/TOOL_USE_SUMMARY.md](#)